LAB 7 · AN INTRODUCTION

Agentic Coding

What it is, what it is good at, and where it will confidently mislead you.



FOR	Anyone who has not watched one of these run before.
READ IT	Before Sunday afternoon. Ten minutes.

What it actually is

An agent is a loop: a model that can call functions, running until it stops asking for them. Agentic coding is that loop with three particular functions wired in - read a file, write a file, run a command.

That is the whole definition. Everything surprising about it follows from one consequence of those three functions: **the loop closes.** The agent can run the thing it just wrote, read what happened, and decide what to do about it. Nothing else in the tooling you have used before can do that.

The difference is easiest to see against the two things people have already met.

Autocomplete



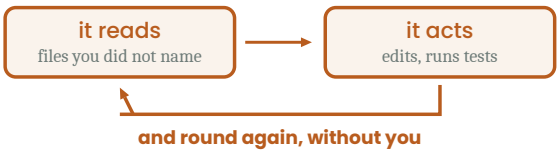
stops at a suggestion.

Chat



stops at an answer. You are still the hands.

Agentic



it observes what happened, then decides what next.

Autocomplete and chat both terminate at you. The third one does not need you on every lap.

> **THE DISTINCTION THAT MATTERS**
>
> Autocomplete and chat make **you** faster at writing code. An agent does work while you are not watching, and comes back with a result. Those are different products with different risks, and conflating them is why people are either disappointed or alarmed.

Three things it is not

Most of the confusion comes from picturing one of the following, all of which are wrong.

Not a smarter autocomplete.	Autocomplete predicts the next few characters you were going to type anyway. An agent decides which files to open.
Not a robot that replaces you.	It cannot tell whether the change was a good idea, only whether the tests went green. Those are very different questions.
Not reliable because it sounds reliable.	Fluency and correctness are unrelated. The most dangerous output is the confident, well-structured, subtly wrong one.

Example 1 – A bug nobody could find

This is the repository you will work on during the lab, so you can check every word of it on Sunday.

The situation. A small billing calculator for a freight company. Eighty lines of Python, written in a hurry two years ago by someone who has since left. No tests. A README that states the business rules clearly. Finance have reported that some invoices "look a bit off" but nobody has narrowed it down.

Everyone has worked somewhere with a file like this. The reason it never gets fixed is not that it is hard - it is that finding the problem means holding the README and the code in your head at the same time and checking them against each other, line by line, for an hour, on a Tuesday, when four other things are on fire.

What happens. One instruction, then four exchanges:

YOU	**Read the README and the code. Where do they disagree?**
RUNS	`reads README.md, rates.py, invoices.py, consignments.csv`
AGENT	Four discrepancies. The clearest: the README says the bulk discount applies to orders of 100 units *or more*, but `apply_bulk_discount` tests `qty > 100`. Two consignments in the sample data are exactly 100 and are being overcharged.
YOU	**Don't fix it yet. Write a test that fails because of that, and show me it failing.**
RUNS	`writes test_rates.py, runs pytest`
OUTPUT	`FAILED test_bulk_discount_at_exactly_100 - assert 1230.00 == 1131.60`
YOU	**Now fix it. Change nothing else.**
RUNS	`edits rates.py line 17, runs pytest`
OUTPUT	`1 passed`

Four minutes, not an hour. But the interesting part is not the speed.

> **ONE DOWN**
>
> That is one of several things wrong with this repository, and it is shown here deliberately so you can see the shape of the work. The others are still there, and at least one of them cannot be found by reading the code at all. Do not assume you have seen the exercise.

Why that exchange was shaped like that

Notice what was *not* said. Nobody asked it to fix anything in the first message. The instruction was "where do they disagree", and the second instruction was explicitly **don't fix it yet**.

That is the discipline the whole lab is built around, and it comes down to one sentence:

> **INSIST ON THIS, EVERY SINGLE TIME**
>
> "The model says there is a bug" and "there is a red test on my screen" are completely different claims. Ask for the failing test **before** the fix. A fix without a failing test is a guess that you have agreed to ship.

The failing test does three jobs at once. It proves the bug is real rather than plausible. It proves the agent understood which bug it was talking about. And when the fix lands, it proves the fix worked - without you reading a single line of the change.

That last point is what makes this scale. You are not reviewing code; you are reviewing *evidence*.

Example 2 - The first day on an unfamiliar codebase

The use case that converts sceptics, because there is nothing to review and nothing to break.

You have joined a team, or inherited a service, or been asked to fix something urgent in a repository you have never opened. Thirty thousand lines. The person who wrote it is on holiday. The documentation is a README from 2023 and a Confluence page that contradicts it.

Normally this costs you two days of reading before you can make a one-line change with any confidence. Instead:

YOU	**I've never seen this repo. What does it do, what are the main moving parts, and where would I start if I needed to change how pricing works?**
RUNS	`reads 40+ files, follows imports, greps for call sites`
AGENT	It is an order pipeline with three entry points. Pricing is calculated in two places that do not agree - `pricing/engine.py` for the web path and an older copy inside `batch/nightly.py`. The web one is the live path; the batch one is used for reconciliation and has drifted. Start in engine.py, but expect reconciliation to break.
YOU	**How do I run the tests?**
RUNS	`reads Makefile, CI config, tries make test`
OUTPUT	`214 passed, 3 skipped in 48s`

Nothing was changed. Nothing could have been changed - you only asked it to read. The risk of this use case is approximately zero, and it compresses the most tedious two days in software engineering into a ten-minute conversation.

For anyone about to start a new job, this is the single most valuable thing in this document.

Where it goes wrong

An introduction that only showed you the good parts would be an advert. Here is the failure mode that actually costs people money.

It is excellent at reading, good at narrow changes, and dangerous at broad ones. Ask for a wide refactor and you will get back something clean, well-structured, plausible, and subtly different in behaviour. The diff will look fine. It will read like something a competent colleague wrote. And the thing it quietly changed is the thing you had no test for.

The reason is not that the model is careless. It is that the loop can only close around things that produce a signal.

Can the agent tell whether it got this right?

YES – IT CLOSES THE LOOP ITSELF

- a test that passes or fails
- code that compiles, or doesn't
- a linter's complaint
- a crash and its stack trace
- output that matches a fixture

NO – ONLY YOU CAN

- whether the rule it inferred is the rule you meant
- whether the refactor preserved behaviour you never tested
- whether the thing it deleted was load-bearing
- whether the bug it did not find is worse
- whether any of this was a good idea

Everything on the left is why it works. Everything on the right is why you read the diff.

There is a second failure worth naming, because you will meet it in the lab. Some bugs are invisible from the code alone - they only appear when you look at the *data* the code runs on. An agent reading source sees a reasonable-looking default and moves on. Telling it to **run** the code on real inputs rather than reason about it is often the difference between finding the bug and confidently reporting that there isn't one.

The working discipline

Four habits. None of them are optional, and all of them take seconds.

Commit before you start.	Work on a branch. `git checkout .` must always be free. Agent work is cheap to redo and expensive to half-review.
Ask for the failing test first.	Every time. No exceptions. Evidence, not assertions.
Keep the task narrow.	"Fix the discount boundary" - not "clean up the billing code". A task you can describe in one clause produces a diff you can read.
Read the whole diff.	If it is too big to read, the task was too big. Go back and split it.

Try it before Sunday

Ten minutes, on a repository you already know, with no risk at all.

Pick any project of your own - or clone something off GitHub you have never read. Then ask it only to *read*:

> **THE FOUR QUESTIONS**
>
> What does this project do, and who is it for?
> What are the main moving parts and how do they fit together?
> Where is the messiest part, and why?
> If I wanted to add X, where would I start?

On a repository you know well, you will be able to grade the answers immediately - which is the point. You will find out where it is sharp and where it is confidently vague, on material where you are the expert. That calibration is worth more than any amount of reading about it, including this document.

Then, if you want the full experience, give it write access on a branch and ask for one small, specific change. Read the diff. Throw it away. Do it again with a better-worded instruction.

What you should take into the lab

Three sentences:

> **ONE**
>
> An agent is a loop around a model that can read files, write files and run commands. The loop closing is the whole difference.

> **TWO**
>
> It is excellent at reading, good at narrow changes, and dangerous at broad ones. The skill is knowing which one you just asked for.

> **THREE**
>
> Ask for the failing test before the fix. You are reviewing evidence, not code.

Aurora Freight is fictional. The billing repository, its README and the bug described in Example 1 are real and are in the lab folder - you can check every claim in this document on Sunday morning.